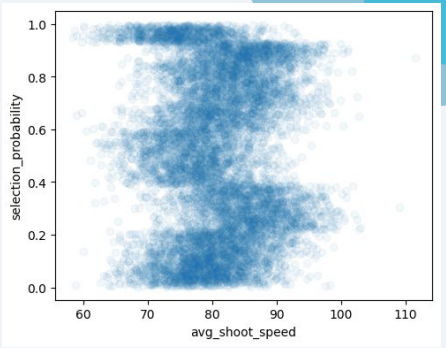
ICE HOCKEY
MACHINE LEARNING

Knowledge Discovery & Data Mining · TU Graz

Group 100

Felix Grabler Muhammad Hanzla Shoukat Simon Tilzer Md Asib Mujtoba Shanto

FEATURE SELECTION

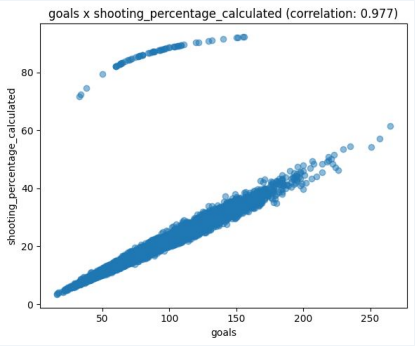


Most important criterion of a feature: **correlation with target column**

- multiple different **correlation metrics**
  - column types (numerical, categorical, boolean)
  - (non-)linear metrics: pearson, spearman, distance correlation, …
- sort by maximum absolute correlation ⇒ 16 features

Many columns with high **inter-feature correlation** may not be beneficial ⇒ prune

- calculate correlation between **all pairs** of feature columns
- discard one column of pair if correlation above **threshold**
- used high threshold of **>= 0.95**, as 16 columns already fine
- drop 2 features ⇒ 14 features left



golden_tipps	0.585	mutual inf.
won_championships	0.247	spearman
training_pct	0.191	mutual inf.
experience_level	0.159	eta squared
goals	0.159	distance corr.

METHODOLOGY

How did we perform our **training**, **validation** and **testing**?

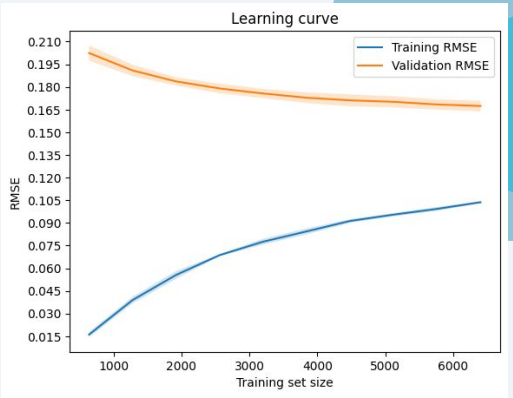
- evaluation metric: **root mean squared error**
- testing:
  - train/test **split** (80%/20%)
  - testing data not used during validation
  - evaluation of test data prediction once per model type
  - shown at the **end of notebook**
- validation:
  - **KFold cross validation** (k=5)
  - would lose more data through another split

What was our general approach to finding a “good” **estimator**?

- baselines: constant, random, linear regression
- selecting 3 model types
  - prepare/select features
  - analyze influence of hyperparameters
  - tune hyperparameters iteratively

model	RMSE	MAE	R2
linear	0.230	0.185	0.431
constant	0.305	0.270	-2.279
random	0.426	0.348	-0.949

EVALUATION 1

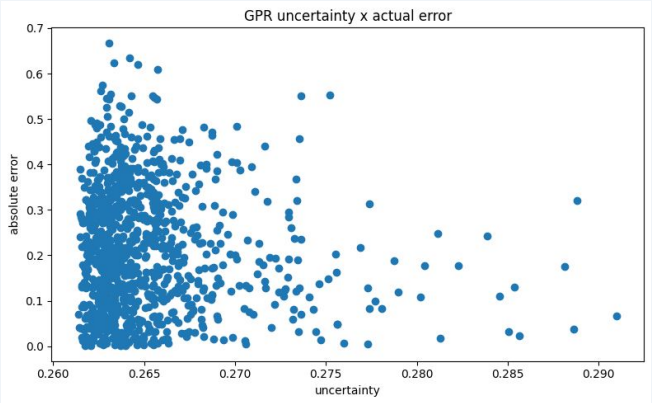


XGBoost

- tree based ⇒ no additional imputation/feature engineering required
- testing influence of **n_estimators** and **learning_rate** on error
  - no big difference between **n_estimators** of 1000 and 2000 anymore
- learning curve: validation flattening, considerable gap between learning/validation error
- ⇒ introduce regularization ⇒ randomized search on possibly helpful hyper parameters
  - validation score got better
  - validation/training error got closer

Gaussian Process Regression (GPR)

- has obvious drawbacks, but wanted to test it
- training runtime initially very bad ⇒
  - select only 4 features (smooth numerical, ordered categorical)
  - only train on sample of 1000 rows
- testing out different kernels: Matern (smooth features), White Kernel (noise)
  - selection improved result
- lin. correlation: **prediction certainty** X error ⇒ very low

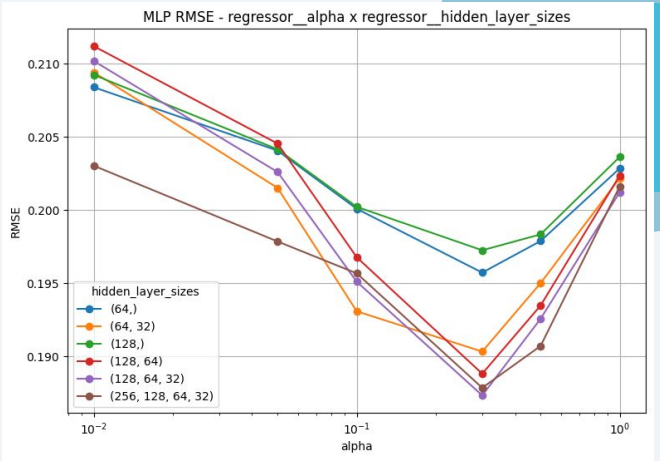


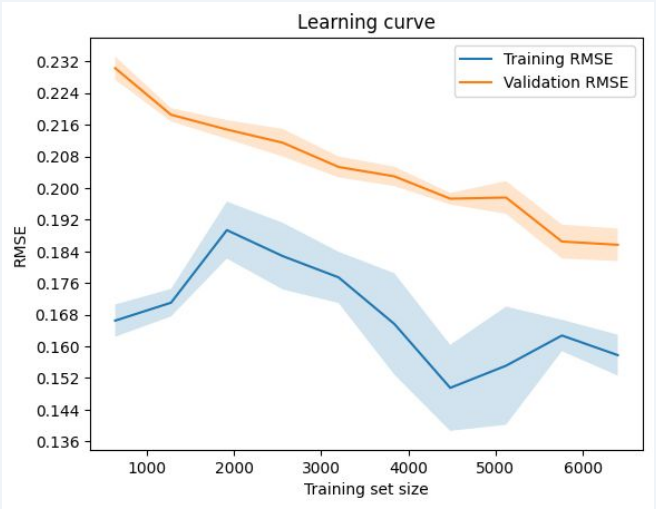
EVALUATION 2

MLP (neural network)

- using all features ⇒ conversion to numeric + scaling
- testing hyperparameters **hidden_layer_sizes** and **alpha**
  - bigger/more hidden layers + alpha sweet spot ⇒ reduced error
- further testing **learning_rate** and **batch_size**
  - trends not as obvious
- testing the **activation_function** and alpha
  - default **relu** does perform best
- learning curve: **falling** validation error ⇒ more data needed

model	RMSE	MAE	R2
XGBoost	0.164	0.124	0.71
MLP	0.187	0.143	0.623
linear	0.230	0.185	0.431
GPR	0.275	0.233	0.186





DISCUSSION

- XGBoost (tree based)
  - is most robust, easy to use and best scoring
- neural network
  - might have been best with more training data
- GPR has performed by far the worst
  - extremely slow training
  - error worse then linear baseline
  - high engineering effort
  - prediction certainty not really reliable
  - might have been better if less training data was available
- our decision: XGBoost
  - considerably better then linear baseline